Design Patterns

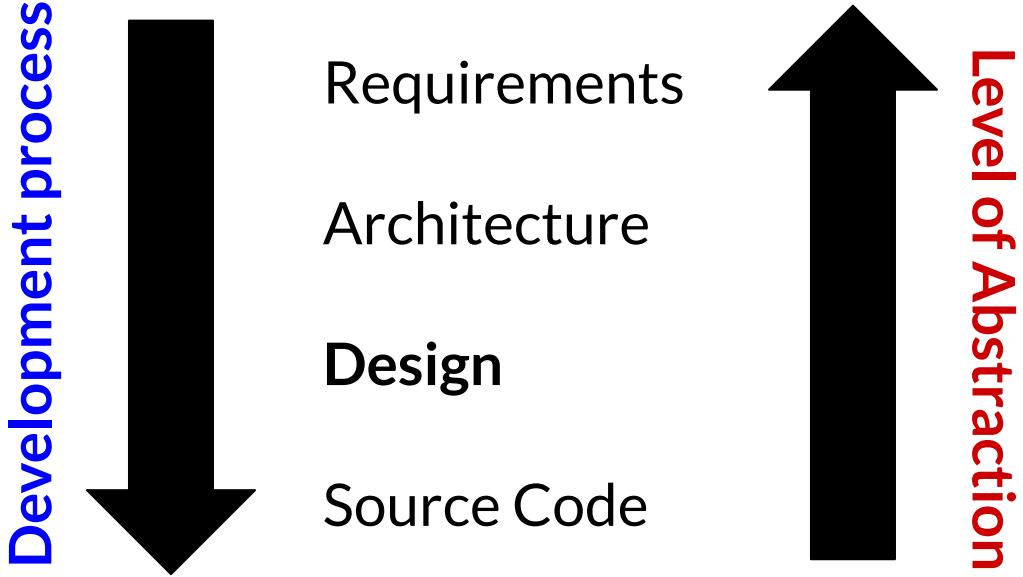
Martin Kellogg

Design Patterns

Today’s agenda:

- **Strategies for good design**
- Examples of design patterns

“Architecture” vs “Design”



Definition: ***software design*** is the structure or organization of a particular component of your system

Design

Key goal: design for **change** and **reuse**

Design

Key goal: design for **change** and **reuse**

- In class, many programs are written once, to a fixed specification, and then **thrown away**

Design

Key goal: design for **change** and **reuse**

- In class, many programs are written once, to a fixed specification, and then **thrown away**
- In industry, many programs are written once and then **modified** as requirements, customers, and developers change

Design

Key goal: design for **change** and **reuse**

- In class, many programs are written once, to a fixed specification, and then **thrown away**
- In industry, many programs are written once and then **modified** as requirements, customers, and developers change
- Many fundamental tenets of object-oriented design facilitate **subsequent change**
    - You may have seen these before, but now you are in a position to really appreciate the motivation!

Design: desiderata

Design: desiderata

- Classes are **open** to be modified/extended without invasive changes

Design: desiderata

- Classes are **open** to be modified/extended without invasive changes
- **Subtype polymorphism** enables changes behind interfaces

Design: desiderata

- Classes are **open** to be modified/extended without invasive changes
- **Subtype polymorphism** enables changes behind interfaces
- Classes **encapsulate** details likely to change behind (small) stable interfaces

Design: desiderata

- Classes are **open** to be modified/extended without invasive changes
- **Subtype polymorphism** enables changes behind interfaces
- Classes **encapsulate** details likely to change behind (small) stable interfaces
- Internal parts can be developed **independently**

Design: desiderata

- Classes are **open** to be modified/extended without invasive changes
- **Subtype polymorphism** enables changes behind interfaces
- Classes **encapsulate** details likely to change behind (small) stable interfaces
- Internal parts can be developed **independently**
- Internal details of other classes do not need to be understood, **contract** is sufficient

Design: desiderata

- Classes are **open** to be modified/extended without invasive changes
- **Subtype polymorphism** enables changes behind interfaces
- Classes **encapsulate** details likely to change behind (small) stable interfaces
- Internal parts can be developed **independently**
- Internal details of other classes do not need to be understood, **contract** is sufficient
- Class implementations and their contracts can be tested separately (**unit testing**)

Design for reuse

Design for reuse: delegation

Definition: ***Delegation*** is when one object relies on another object for some subset of its functionality

- e.g., in Java, Sort delegates functionality to some Comparator

Design for reuse: delegation

Definition: ***Delegation*** is when one object relies on another object for some subset of its functionality

- e.g., in Java, Sort delegates functionality to some Comparator

Judicious delegation enables **code reuse**:

Design for reuse: delegation

Definition: ***Delegation*** is when one object relies on another object for some subset of its functionality

- e.g., in Java, Sort delegates functionality to some Comparator

Judicious delegation enables **code reuse**:

- Sort can be reused with arbitrary sort orders

Design for reuse: delegation

Definition: ***Delegation*** is when one object relies on another object for some subset of its functionality

- e.g., in Java, Sort delegates functionality to some Comparator

Judicious delegation enables **code reuse**:

- Sort can be reused with arbitrary sort orders
- Comparators can be reused with arbitrary client code that needs to compare integers

Design for reuse: delegation

Definition: ***Delegation*** is when one object relies on another object for some subset of its functionality

- e.g., in Java, Sort delegates functionality to some Comparator

Judicious delegation enables **code reuse**:

- Sort can be reused with arbitrary sort orders
- Comparators can be reused with arbitrary client code that needs to compare integers
- Reduce “cut and paste” code and defects

Design for reuse: delegation

Definition: ***Delegation*** is when one object relies on another object for some subset of its functionality

- e.g., in Java, Sort delegates functionality to some Comparator

Judicious delegation enables **code**

- Sort can be reused with ar
- Comparators can be reuse needs to compare integers
- Reduce "cut and paste" cod

Java's Comparator class is an example of a ***design pattern*** (the **template method pattern**) that can be reused in other settings

Design for Extensibility: Contracts + Subtyping

Design for Extensibility: Contracts + Subtyping

- **Design by contract** prescribes that software designers should define formal, precise and **verifiable** interface specifications for components, which extend the ordinary definition of abstract data types with **preconditions, postconditions** and invariants

Design for Extensibility: Contracts + Subtyping

- **Design by contract** prescribes that software designers should define formal, precise and **verifiable** interface specifications for components, which extend the ordinary definition of abstract data types with **preconditions, postconditions** and invariants
- A subclass can only have **weaker** preconditions
  - My super only works on positive numbers, but I work on all numbers

Design for Extensibility: Contracts + Subtyping

- **Design by contract** prescribes that software designers should define formal, precise and **verifiable** interface specifications for components, which extend the ordinary definition of abstract data types with **preconditions, postconditions** and invariants
- A subclass can only have **weaker** preconditions
  - My super only works on positive numbers, but I work on all numbers
- A subclass can only have **stronger** postconditions
  - My super returns any shape, but I return squares

Design for Extensibility: Contracts + Subtyping

- **Design by contract** prescribes that software designers should define formal, precise and **verifiable** interface specifications for components, which extend the ordinary definition of abstract data types with **preconditions, post**
- A subclass can only have **weake**
  - My super only works on pos numbers
- A subclass can only have **stronger** **postconditions**
  - My super returns any shape, but I return squares

This is called the ***Liskov Substitution Principle***: “any subclass object should be safe to use in place of a superclass object at run time”

Design to Admit Testing

Design to Admit Testing

- Consider a ***library-oriented architecture***, a variation of **modular programming** or **microservice architecture** with a focus on separation of concerns and **interface design**

Design to Admit Testing

- Consider a ***library-oriented architecture***, a variation of **modular programming** or **microservice architecture** with a focus on separation of concerns and **interface design**
  - “Package logical components of your application independently - literally as separate gems, eggs, RPMs, or whatever - and maintain them as internal open-source projects … This approach combats the tightly-coupled spaghetti so often lurking in big codebases by giving everything the Right Place in which to exist.”

Design to Admit Unit Testing

- Recall: it is hard to generate test inputs with high coverage for areas "**deep inside**" the code
  - Must solve the constraints for main(), then for foo(), then for bar(), etc., all at the same time!

Design to Admit Unit Testing

- Recall: it is hard to generate test inputs with high coverage for areas "**deep inside**" the code
  - Must solve the constraints for main(), then for foo(), then for bar(), etc., all at the same time!
- The farther code is from an entry point, the harder it is to test
  - This is one of the motivations behind **unit testing**

Design to Admit Unit Testing

- Recall: it is hard to generate test inputs with high coverage for areas "**deep inside**" the code
  - Must solve the constraints for main(), then for foo(), then for bar(), etc., all at the same time!
- The farther code is from an entry point, the harder it is to test
  - This is one of the motivations behind **unit testing**
- Solution: design with **more entry points** for self-contained functionality (cf. AVL tree, priority queue, etc.)

Example: MVC + Angry Birds



- Suppose you are designing Angry Birds
- It's a game, and also a simulation, so MVC is a reasonable choice

Example: MVC + Angry Birds



- Suppose you are designing Angry Birds
- It's a game, and also a simulation, so MVC is a reasonable choice
- Design so that it can be tested without someone actually playing the game!

Example: MVC + Angry Birds



- Suppose you are designing Angry Birds
- It's a game, and also a simulation, so MVC is a reasonable choice
- Design so that it can be tested without someone actually playing the game!
    - e.g., have an **interface** where abstract commands can be queued up: one way to get them is from the UI, but another is programmatic

Example: MVC + Angry Birds



- Suppose you are designing Angry Birds
- It's a game, and also a simulation, so MVC is a reasonable choice
- Design so that it can be tested without someone actually playing the game!
  - e.g., have an **interface** where abstract commands can be queued up: one way to get them is from the UI, but another is programmatic
  - “If I create a world with blocks X, Y and Z and then we launch bird A at angle B, does C occur within five timesteps?”

Example: fault injection

- Microsoft's Driver Verifier sat between a driver and the operating system and “pretended to fail (some of the time)” to expose poor driver code
- The CHESS project sat between a program and the scheduler and “forced strange schedules” to expose poor concurrency code

Example: fault injection

- Microsoft's Driver Verifier sat between a driver and the operating system and “pretended to fail (some of the time)” to expose poor driver code
- The CHESS project sat between a program and the scheduler and “forced strange schedules” to expose poor concurrency code
- Problem for both: Hardware, OS and Networking errors can occur **infrequently**, but you still want to test them

Example: fault injection

- Microsoft's Driver Verifier sat between a driver and the operating system and “pretended to fail (some of the time)” to expose poor driver code
- The CHESS project sat between a program and the scheduler and “forced strange schedules” to expose poor concurrency code
- Problem for both: Hardware, OS and Networking errors can occur **infrequently**, but you still want to test them
    - Must design for it! But how…?

Example: fault injection: add a level of indirection

- **Old adage**: the solution to everything in computer science is either to add a level of indirection or to add a cache

Example: fault injection: add a level of indirection

- **Old adage**: the solution to everything in computer science is either to add a level of indirection or to add a cache
- Don't have your code call fopen() or cout or whatever directly

Example: fault injection: add a level of indirection

- **Old adage**: the solution to everything in computer science is either to add a level of indirection or to add a cache
- Don't have your code call fopen() or cout or whatever directly
- Instead, add a very thin **level of indirection** where you call my_fopen which then calls fopen

Example: fault injection: add a level of indirection

- **Old adage**: the solution to everything in computer science is either to add a level of indirection or to add a cache
- Don't have your code call fopen() or cout or whatever directly
- Instead, add a very thin **level of indirection** where you call my_fopen which then calls fopen
- Later add “if coin_flip() then fail else ...” to that indirection layer to **inject faults** while testing
    - let the compiler optimize it away for your production code

Designing for testing: tests for free

Designing for testing: tests for free

- Many programs transform data from one format to another (cf. adapter pattern)

Designing for testing: tests for free

- Many programs transform data from one format to another (cf. adapter pattern)
- If the program is implementing a function with similar domain and range, you can often get high-coverage tests “for free” by **composing the program with itself**

Designing for testing: tests for free

- Many programs transform data from one format to another (cf. adapter pattern)
- If the program is implementing a function with similar domain and range, you can often get high-coverage tests “for free” by **composing the program with itself**
  - If possible, design your program so that this is possible

Designing for testing: tests for free

- Many programs transform data from one format to another (cf. adapter pattern)
- If the program is implementing a function with similar domain and range, you can often get high-coverage tests “for free” by **composing the program with itself**
  - If possible, design your program so that this is possible
- Examples:

Designing for testing: tests for free

- Many programs transform data from one format to another (cf. adapter pattern)
- If the program is implementing a function with similar domain and range, you can often get high-coverage tests “for free” by **composing the program with itself**
  - If possible, design your program so that this is possible
- Examples:
  - **Inversion**: forall X. unzip(zip(X)) = X

Designing for testing: tests for free

- Many programs transform data from one format to another (cf. adapter pattern)
- If the program is implementing a function with similar domain and range, you can often get high-coverage tests “for free” by **composing the program with itself**
  - If possible, design your program so that this is possible
- Examples:
  - **Inversion**: forall X. unzip(zip(X)) = X
  - **Convergence**: forall X. sort(sort(X)) = sort(X)

Designing for testing: tests for free

- Many programs transform data from one format to another (cf. adapter pattern)
- If the program is impl
  range, you can often g
  the program with itse
  - If possible, design your program so that this is possible
- Examples:
  - **Inversion**: forall X. unzip(zip(X)) = X
  - **Convergence**: forall X. sort(sort(X)) = sort(X)

These are examples of ***metamorphic testing***, which is a generalization of differential testing. Ask me more in OH!

Design Patterns

Today’s agenda:

- Strategies for good design
- **Examples of design patterns**

Design Patterns

Definition: A ***software design pattern*** is a general, reusable solution to a commonly occurring problem within a given context in software design.

Design Patterns

Definition: A ***software design pattern*** is a general, reusable solution to a commonly occurring problem within a given context in software design.

- patterns reduce surprise for code readers

Design Patterns

Definition: A ***software design pattern*** is a general, reusable solution to a commonly occurring problem within a given context in software design.

- patterns reduce surprise for code readers
- patterns separate the structure of a system from the implementation details

Design Patterns

Definition: A ***software design pattern*** is a general, reusable solution to a commonly occurring problem within a given context in software design.

- patterns reduce surprise for code readers
- patterns separate the structure of a system from the implementation details
- patterns apply across languages

Design Patterns

Definition: A ***software design pattern*** is a general, reusable solution to a commonly occurring problem within a given context in software design.

- patterns reduce surprise for code readers
- patterns separate the structure of a system from the implementation details
- patterns apply across languages
- all patterns have **tradeoffs**. Many common design patterns trade **verbosity or efficiency** for **extensibility**

Design Patterns: non-software

- Design patterns are **common in almost every field** (not just SE)

Design Patterns: non-software

- Design patterns are **common in almost every field** (not just SE)
  - e.g., “multiple choice question” is a design pattern for exam making, “verse-chorus-verse-chorus-bridge-chorus” is a design pattern for songs

Design Patterns: non-software

- Design patterns are **common in almost every field** (not just SE)
  - e.g., “multiple choice question” is a design pattern for exam making, “verse-chorus-verse-chorus-bridge-chorus” is a design pattern for songs
- *The Design of Everyday Things* (famous book): design serves as the **communication** between object and user

Design Patterns: non-software

- Design patterns are **common in almost every field** (not just SE)
  - e.g., “multiple choice question” is a design pattern for exam making, “verse-chorus-verse-chorus-bridge-chorus” is a design pattern for songs
- *The Design of Everyday Things* (famous book): design serves as the **communication** between object and user
  - although people often blame themselves when objects appear to malfunction, it is not the fault of the user but rather the lack of intuitive guidance that should be present in the design

Design Patterns: non-software

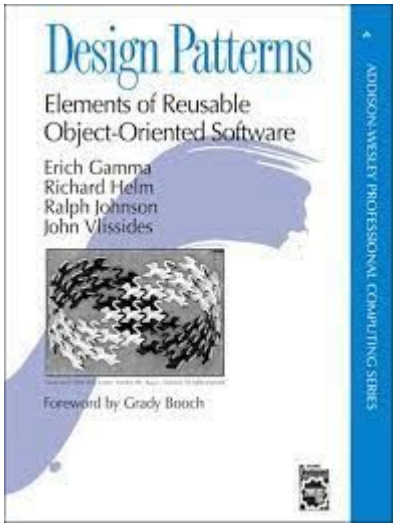
- Design patterns are **commo**
  - e.g., “multiple choice que
    making, “verse-chorus-v
    pattern for songs
- *The Design of Everyday Thing*
  communication between object and user
  - although people often blame themselves when objects appear to malfunction, it is not the fault of the user but rather the lack of intuitive guidance that should be present in the design

Same ideas apply to software:

- design GUIs that people **intuitively** know how to use
- design code that other developers **intuitively** know how to read

Design Patterns: “gang of four”

- The book popularizing software design patterns is often called the “**Gang of Four**” book after its four authors
- I don’t care if you remember this, but it’ll be handy to know about (e.g., for interviews)



Design Patterns: high-level advice

Design Patterns: high-level advice

- Consider **code change as a certainty**
  - Redesign is expensive. Choosing the right pattern helps avoid it.

Design Patterns: high-level advice

- Consider **code change as a certainty**
  - Redesign is expensive. Choosing the right pattern helps avoid it.
- Consider your **requirements and their changes**
  - Use patterns that fit your current or anticipated needs.

Design Patterns: high-level advice

- Consider **code change as a certainty**
  - Redesign is expensive. Choosing the right pattern helps avoid it.
- Consider your **requirements and their changes**
  - Use patterns that fit your current or anticipated needs.
- Consider **multiple designs**
  - Diagram your designs before writing code.

Example: Adapter Pattern

- The ***adapter design pattern*** is a structural design pattern that converts the interface of a class into another interface clients expect.

Example: Adapter Pattern

- The ***adapter design pattern*** is a structural design pattern that converts the interface of a class into another interface clients expect.
  - analogy: dongles that convert HDMI to USB-C

Example: Adapter Pattern

- The ***adapter design pattern*** is a structural design pattern that converts the interface of a class into another interface clients expect.
  - analogy: dongles that convert HDMI to USB-C
- Examples:

Example: Adapter Pattern

- The ***adapter design pattern*** is a structural design pattern that converts the interface of a class into another interface clients expect.
  - analogy: dongles that convert HDMI to USB-C
- Examples:
  - Implementing a Stack interface using a LinkedList interface

Example: Adapter Pattern

- The ***adapter design pattern*** is a structural design pattern that converts the interface of a class into another interface clients expect.
  - analogy: dongles that convert HDMI to USB-C
- Examples:
  - Implementing a Stack interface using a LinkedList interface
  - Early implementations of fstream in C++
    - … were simply adapters around the C FILE macro

Example: Adapter Pattern

- The ***adapter design pattern*** is a structural design pattern that converts the interface of a class into another interface clients expect.
  - analogy: dongles that
- Examples:
  - Implementing a Stack
  - Early implementation
    - … were simply adapters around the STL list/vector

Structural design patterns like the adapter pattern ease design by identifying simple ways to realize **relationships** among **entities**.

Other structural design patterns

Other structural design patterns

- The **composite design pattern** allows clients to treat individual objects and groups of objects uniformly

Other structural design patterns

- The **composite design pattern** allows clients to treat individual objects and groups of objects uniformly
  - e.g., selecting and moving objects in PowerPoint

Other structural design patterns

- The **composite design pattern** allows clients to treat individual objects and groups of objects uniformly
  - e.g., selecting and moving objects in PowerPoint
- The **proxy design pattern** provides a surrogate or placeholder for another object to control access to it

Other structural design patterns

- The **composite design pattern** allows clients to treat individual objects and groups of objects uniformly
  - e.g., selecting and moving objects in PowerPoint
- The **proxy design pattern** provides a surrogate or placeholder for another object to control access to it
  - e.g., std::vector exposes std::vector::reference as a method of accessing individual bits. In particular, objects of this class are returned by operator[] by value. (https://en.cppreference.com/w/cpp/container/vector_bool)

Creational design patterns

Creational design patterns

- ***Creational design patterns*** control object creation so that objects are created in a manner suitable for the situation. They make a system **independent of how its objects are created**.

Creational design patterns

- ***Creational design patterns*** control object creation so that objects are created in a manner suitable for the situation. They make a system **independent of how its objects are created**.
- A plain constructor **may not allow** you to:
  - Control how and when an object is used
  - Overcome language limitations (e.g., no default arguments)
  - Hide polymorphic types
  - Specify different combinations of optional arguments

Creational design patterns

- ***Creational design patterns*** control object creation so that objects are created in a manner su
  system **independent of ho**
- A plain constructor **may n**
  - Control how and when an object is used
  - Overcome language limitations (e.g., no default arguments)
  - Hide polymorphic types
  - Specify different combinations of optional arguments

Different creational patterns allow you to overcome these limitations of simple constructors

Example: factory pattern

- Suppose we need to create and use polymorphic objects **without exposing their types** to the client

Example: factory pattern

- Suppose we need to create and use polymorphic objects **without exposing their types** to the client
  - Recall: design for maintainability and extensibility. We don't want the client to depend on (and thus “lock in”) the actual subtypes.

Example: factory pattern

- Suppose we need to create and use polymorphic objects **without exposing their types** to the client
  - Recall: design for maintainability and extensibility. We don't want the client to depend on (and thus “lock in”) the actual subtypes.
- The typical solution is to write a function (a ***factory method***) that creates objects of the type we want but returns that object so that it appears to be (“cast to”) a member of the base class

Example: factory pattern

- The ***factory method pattern*** (or just ***factory pattern***) is a creational design pattern that uses factory methods to create objects without having the return type reveal the exact subclass created.

Example: factory pattern

- The ***factory method pattern*** (or just ***factory pattern***) is a creational design pattern that uses factory methods to create objects without having the return type reveal the exact subclass created.

```
Payment * payment_factory(string name, string type) {
 if (type == “credit_card”)
   return new CreditCardPayment(name);
 else if (type == “bitcoin”)
   return new BitcoinPayment(name);
 … }

Payment * webapp_session_payment =
 payment_factory(customer_name, “credit_card”);
```

Example: factory pattern

- The ***factory method pattern*** (or
  design pattern that uses facto
  without having the return type

Note how the implementation details are hidden from the client, and they can only treat the result as a **generic** payment

```
Payment * payment_factory(string name, string type) {
 if (type == "credit_card")
   return new CreditCardPayment(name);
 else if (type == "bitcoin")
   return new BitcoinPayment(name);
 … }

Payment * webapp_session_payment =
 payment_factory(customer_name, "credit_card");
```

Example: factory pattern

- Suppose we're implementing a computer game with a **polymorphic Enemy class hierarchy**, and we want to spawn **different versions** of enemies based on the difficulty level.
- e.g., normal difficulty = regular Goomba



- hard difficulty = spiked Goomba



Aside: anti-patterns

- An ***anti-pattern*** is a common response to a recurring problem that is usually ineffective and risks being counterproductive.

Aside: anti-patterns

- An ***anti-pattern*** is a common response to a recurring problem that is usually ineffective and risks being counterproductive.
- A bad solution (i.e., anti-pattern) would be to check the difficulty at each of the many places in the code related to spawning enemies:

```
Enemy* goomba = nullptr;
if (difficulty == “normal”)
  goomba = new Goomba();
else if (difficulty == “hard”)
  goomba = new SpikedGoomba();
```

Aside: anti-patterns

- An ***anti-pattern*** is a common response to a recurring problem that is usually ineffective and risks being counterproductive.
- A bad solution (i.e., anti-pattern) would be to check the difficulty at each of the many places in the code related to spawning enemies:

```
Enemy* goomba = nullptr;
if (difficulty == “normal”)
  goomba = new Goomba();
else if (difficulty == “hard”)
  goomba = new SpikedGoomba();
```

Why is this bad?

Aside: anti-patterns

- An ***anti-pattern*** is a common response to a recurring problem that is usually ineffective and risks being counterproductive.
- A bad solution (i.e., anti-pattern) would be to check the difficulty at each of the many places in the code related to spawning enemies:

```
Enemy* goomba = nullptr;
if (difficulty == “normal”)
  goomba = new Goomba();
else if (difficulty == “hard”)
  goomba = new SpikedGoomba();
```

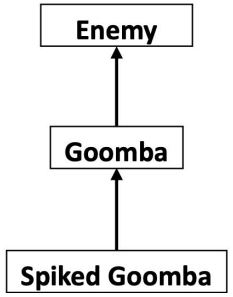
Why is this bad?

- code duplication
- consider how you’d add a new difficulty level…

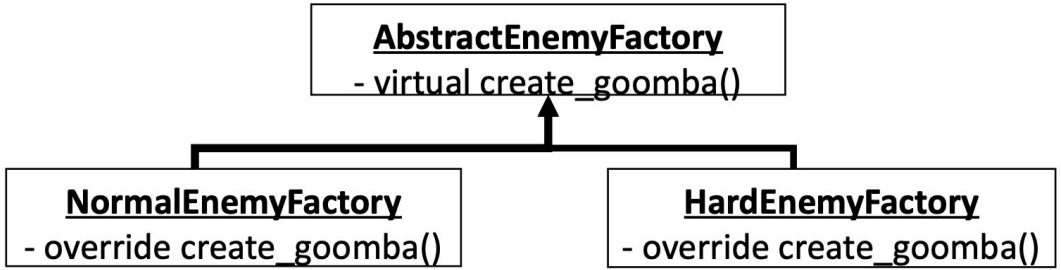
Factory pattern: abstract factories

- The ***abstract factory pattern*** encapsulates a group of factories that have a common theme without specifying their concrete classes.

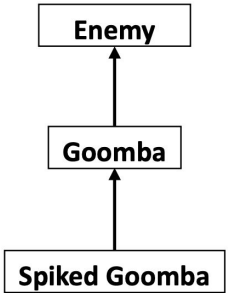
Factory pattern: abstract factories



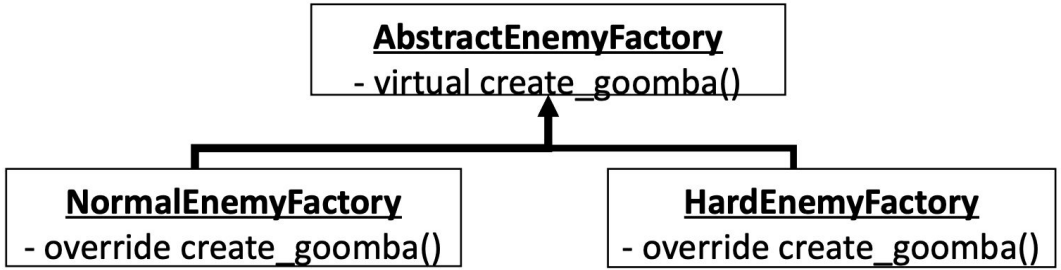
- The ***abstract factory pattern*** encapsulates a group of factories that have a common theme without specifying their concrete classes.



Factory pattern: abstract factories



- The ***abstract factory pattern*** encapsulates a group of factories that have a common theme without specifying their concrete classes.




```
// Only have to do this once!
AbstractEnemyFactory* factory = nullptr;
if (difficulty == “normal”)
  factory = new NormalEnemyFactory();
else if (difficulty == “hard”)
  factory = new HardEnemyFactory();
Enemy* goomba = factory->create_goomba();
```

Scenario: global application state

Scenario: global application state

- Suppose we have some application **state that needs to be globally accessible**. However, we need to control how that data is accessed and updated.

Scenario: global application state

- Suppose we have some application **state that needs to be globally accessible**. However, we need to control how that data is accessed and updated.
- The anti-pattern (**bad**) solution is to have an **unprotected global variable** (e.g., a public static field).

Scenario: global application state

- Suppose we have some application **state that needs to be globally accessible**. However, we need to control how that data is accessed and updated.
- The anti-pattern (**bad**) solution is to have an **unprotected global variable** (e.g., a public static field).
  - fails to control access or updates!

Scenario: global application state

- Suppose we have some application **state that needs to be globally accessible**. However, we need to control how that data is accessed and updated.
- The anti-pattern (**bad**) solution is to have an **unprotected global variable** (e.g., a public static field).
  - fails to control access or updates!
- A “less bad” solution is to put all of the state in one class and have a **global instance** of that class.

Scenario: global application state

- Global variables are usually a **poor design choice**. However:

Scenario: global application state

- Global variables are usually a **poor design choice**. However:
  - If you **must** access some state everywhere, passing it as a parameter to every function clutters the code (readability vs. …)

Scenario: global application state

- Global variables are usually a **poor design choice**. However:
  - If you **must** access some state everywhere, passing it as a parameter to every function clutters the code (readability vs. …)
    - This is not an argument for using global variables to avoid passing a few parameters.

Scenario: global application state

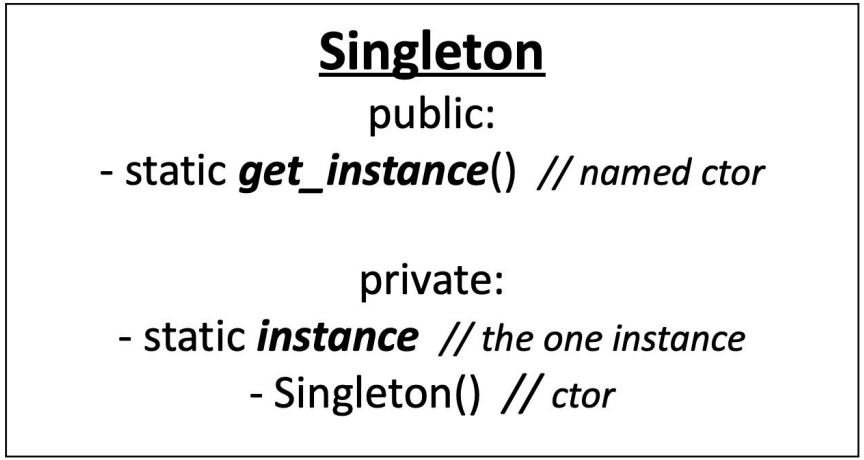
- Global variables are usually a **poor design choice**. However:
  - If you **must** access some state everywhere, passing it as a parameter to every function clutters the code (readability vs. …)
    - This is not an argument for using global variables to avoid passing a few parameters.
  - Or if you need to access state stored outside your program (e.g., database, web API)

Scenario: global application state

- Global variables are usually a **poor design choice**. However:
  - If you **must** access some state everywhere, passing it as a parameter to every function clutters the code (readability vs. …)
    - This is not an argument for using global variables to avoid passing a few parameters.
  - Or if you need to access state stored outside your program (e.g., database, web API)
  - Then global variables **may** be acceptable

Singleton design pattern

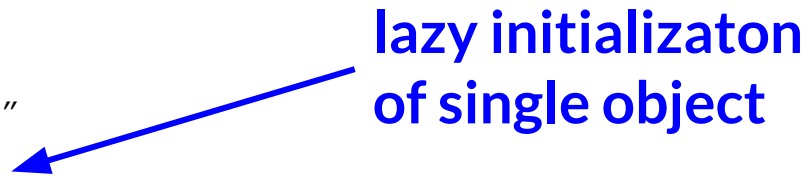
- The ***singleton pattern*** restricts the instantiation of a class to **exactly one** logical instance. It ensures that a class has only one logical instance at runtime and provides a global point of access to it.



Singleton design pattern: example

```
class Singleton {
 // public way to get “the one logical instance”
 public static Singleton get_instance() {
   if (Singleton.instance == null) Singleton.instance = new Singleton();
   return Singleton.instance;
 }
 private static Singleton instance = null;
 private Singleton() { // only runs once
   billing_database = 0;
   System.out.println("Singleton DB created");
 }
 // Our global state
 private int billing_database;
 public int get_billing_count() { return billing_database; }
 public void increment_billing_count() { billing_database += 1; }
}
```

Singleton design pattern: example




```
class Singleton {
 // public way to get "the one logical instance"
 public static Singleton get_instance() {
   if (Singleton.instance == null) Singleton.instance = new Singleton();
   return Singleton.instance;
 }
 private static Singleton instance = null;
 private Singleton() { // only runs once
   billing_database = 0;
   System.out.println("Singleton DB created");
 }
 // Our global state
 private int billing_database;
 public int get_billing_count() { return billing_database; }
 public void increment_billing_count() { billing_database += 1; }
}
```

Singleton design pattern: example

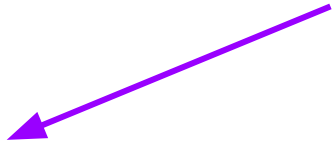
```
class Singleton {
 // public way to get “the one logical instance”
 public static Singleton get_instance() {
   if (Singleton.instance == null) Singleton.instance = new Singleton();
   return Singleton.instance;
 }
 private static Singleton instance = null;
 private Singleton() { // only runs once
   billing_database = 0;
   System.out.println("Singleton DB created");
 }
 // Our global state
 private int billing_database;
 public int get_billing_count() { return billing_database; }
 public void increment_billing_count() { billing_database += 1; }
}
```



this constructor can’t be called any other way

Singleton design pattern: example

```
class Singleton {
 // public way to get “the one logical instance”
 public static Singleton get_instance() {
   if (Singleton.instance == null) Singleton.instance = new Singleton();
   return Singleton.instance;
 }
 private static Singleton instance = null;
 private Singleton() { // only runs once
   billing_database = 0;
   System.out.println("Singleton DB created");
 }
 // Our global state
 private int billing_database;
 public int get_billing_count() { return billing_database; }
 public void increment_billing_count() { billing_database += 1; }
}
```



all clients share this global state

Singleton design pattern: example

What is the output of this code?

```
class Main {
 public static void main(String[] args) {
   int bills = Singleton.get_instance().get_billing_count();
   System.out.println(bills);

   Singleton.get_instance().increment_billing_count();
   bills = Singleton.get_instance().get_billing_count();
   System.out.println(bills);
 }
}
```

Singleton

public:

- static ***get_instance***() *// named ctor*
- get_billing_count()
- increment_billing_count() *// adds 1*

private:

- static ***instance*** *// the one instance*
- Singleton() *// ctor, prints message*
- billing_database

Singleton design pattern: example

What is the output of this code?

```
class Main {
 public static void main(String[] args) {
   int bills = Singleton.get_instance().get_billing_count();
   System.out.println(bills);

   Singleton.get_instance().increment_billing_count();
   bills = Singleton.get_instance().get_billing_count();
   System.out.println(bills);
 }
}
```

Singleton

public:

- static ***get_instance***() *// named ctor*
- get_billing_count()
- increment_billing_count() *// adds 1*

private:

- static ***instance*** *// the one instance*
- Singleton() *// ctor, prints message*
- billing_database

Output:

```
Singleton DB created
0
1
```

Singleton design pattern: get_instance()

- Could we avoid typing Single.get_instance() so many times by doing this at all of the points in our program that use the singleton?

```
Single s = Singleton.get_instance();
System.out.println(s.get_billing_count());
… // later
System.out.println(s.get_billing_count());
```

Singleton design pattern: get_instance()

- Could we avoid typing Single.get_instance() so many times by doing this at all of the points in our program that use the singleton?

```
Single s = Singleton.get_instance();
System.out.println(s.get_billing_count());
… // later
System.out.println(s.get_billing_count());
```

- Is this a good idea or not?

Singleton design pattern: get_instance()

- Could we avoid typing Single.get_instance() so many times by doing this at all of the points in our program that use the singleton?

```
Single s = Singleton.get_inst
System.out.println(s.get_bill
… // later
System.out.println(s.get_bill
```

- Is this a good idea or not?

This is a **bad idea**. There is **no guarantee** that get_instance() will return the same pointer (same object) every time it is called. (It may return different **concrete copies** of the **same logical item**.)

Singleton design pattern: another example

- Suppose we are implementing a computer version of the card game Euchre. In addition to a few abstract datatypes, we have a Game class that stores the state needed for a game of Euchre. When started, our application prototype plays one game of Euchre and then exits.

- Design question: **should we make Game a singleton**?

Singleton design pattern: another example

- Making Game a Singleton is **tempting**
  - There is only one Game instance in our application

Singleton design pattern: another example

- Making Game a Singleton is **tempting**
  - There is only one Game instance in our application
- However, there only **happens** to be one instance of Game. There's **no requirement** that we only have one instance.

Singleton design pattern: another example

- Making Game a Singleton is **tempting**
  - There is only one Game instance in our application
- However, there only **happens** to be one instance of Game. There's **no requirement** that we only have one instance.
- We should only use the Singleton pattern when current or future **requirements** dictate that only one instance should exist.

Singleton design pattern: another example

- Making Game a Singleton is **tempting**
  - There is only one Game instance in our application
- However, there only **happens** to be one instance of Game. There's **no requirement** that we only have one instance.
- We should only use the Singleton pattern when current or future **requirements** dictate that only one instance should exist.
  - Singleton is **not** a license to make everything global.

Scenario: binge-watching

- Suppose we're implementing a video streaming website in which users can “binge-watch” (or “lock on”) to one channel. The user will then see that channel's videos in sequence. When the last such video is watched, the user should stop binge-watching that channel.

Scenario: binge-watching

- Idea: when the last video is watched, call release_binge_watch() on the user.

Scenario: binge-watching

- Idea: when the last video is watched, call release_binge_watch() on the user.

```
class User {
 public void release_binge_watch(Channel c) {
   if (c == binge_channel) {
     binge_channel = null;
   }
 }
 private Channel binge_channel;
}
```

Scenario: binge-watching

- Idea: when the last video is watched, call release_binge_watch() on the user.

```
class User {
 public void release_binge_watch(Channel c) {
   if (c == binge_channel) {
     binge_channel = null;
   }
 }
 private Channel binge_channel;
}
```

```
class Channel {
 // Called when the last video is shown
 public void on_last_video_shown() {
   // Global accessor for the user
   get_user().release_binge_watch(this);
 }
}
```

Scenario: binge-watching

- Idea: when the last video is watched, call release_binge_watch() on the user.

```
class User {
 public void release_binge_watch(Channel c) {
   if (c == binge_channel) {
     binge_channel = null;
   }
 }
 private Channel binge_channel;
}
```

```
class Channel {
 // Called when the last video is shown
 public void on_last_video_shown() {
   // Global accessor for the user
   get_user().release_binge_watch(this);
 }
}
```

- What are some problems with this approach?

Scenario: binge-watching: anti-patterns

- With this design, User and Channel are **tightly coupled**
  - Changing one likely requires a change to the other

Scenario: binge-watching: anti-patterns

- With this design, User and Channel are **tightly coupled**
  - Changing one likely requires a change to the other
- The design does not support multiple users

Scenario: binge-watching: anti-patterns

- With this design, User and Channel are **tightly coupled**
  - Changing one likely requires a change to the other
- The design does not support multiple users
- What if we later want to update a user's “recommendation queue” when they finish binge-watching a channel?

Scenario: binge-watching: anti-patterns

- With this design, User and Channel are **tightly coupled**
  - Changing one likely requires a change to the other
- The design does not support multiple users
- What if we later want to update a user's “recommendation queue” when they finish binge-watching a channel?
- Whenever requirements change and we want to do something else when a video finishes (e.g., update advertising) we **must update the Channel class** and couple it to the new feature

Scenario: binge-watching: anti-patterns

- With this design, User and Channel are **tightly coupled**
  - Changing one likely requires a change to the other
- The design does n
- What if we later w mendation queue" when they finish binge-watching a channel?
- Whenever requirements change and we want to do something else when a video finishes (e.g., update advertising) we **must update the Channel class** and couple it to the new feature

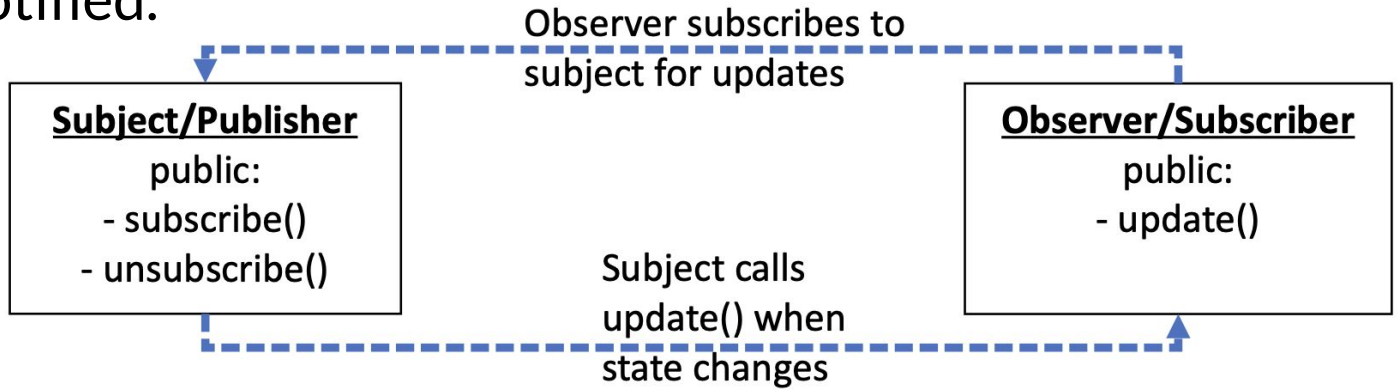
What can we do instead?

Observer Pattern

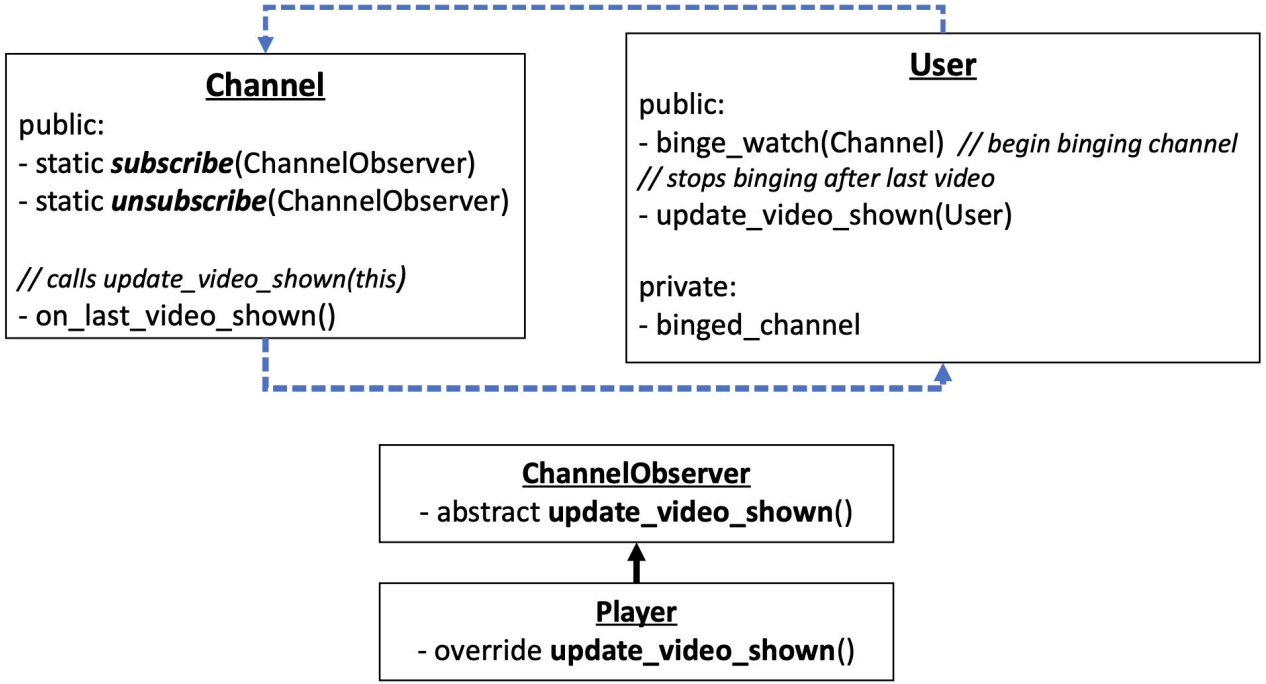
- The ***observer pattern*** (also called "***publish-subscribe***") allows dependent objects to be notified automatically when the state of a subject changes. It defines a one-to-many dependency between objects so that when one object changes state, all of it dependents are notified.

Observer Pattern

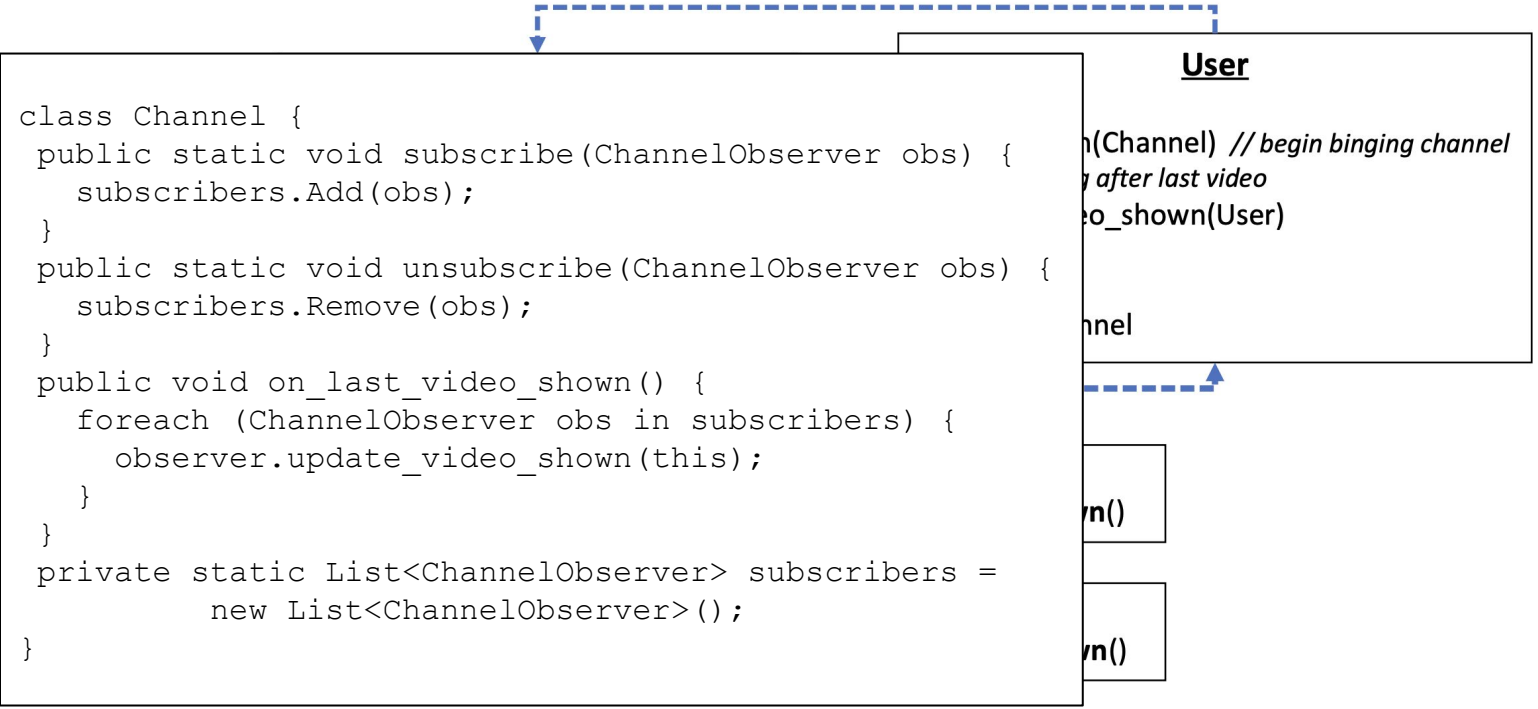
- The ***observer pattern*** (also called "***publish-subscribe***") allows dependent objects to be notified automatically when the state of a subject changes. It defines a one-to-many dependency between objects so that when one object changes state, all of it dependents are notified.



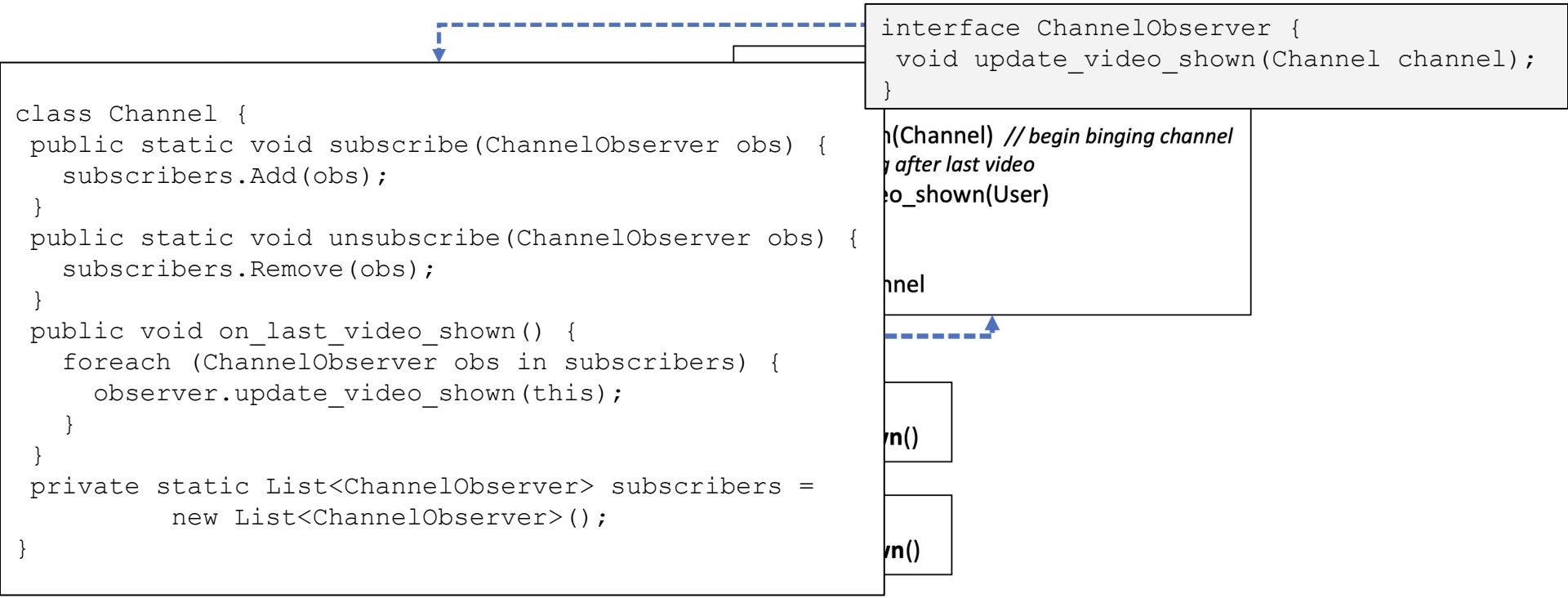
Observer Pattern: bing-watch scenario



Observer Pattern: bing-watch scenario



Observer Pattern: bing-watch scenario



```
interface ChannelObserver {
 void update_video_shown(Channel channel);
}
```

```
class Channel {
 public static void subscribe(ChannelObserver obs) {
   subscribers.Add(obs);
 }
 public static void unsubscribe(ChannelObserver obs) {
   subscribers.Remove(obs);
 }
 public void on_last_video_shown() {
   foreach (ChannelObserver obs in subscribers) {
     observer.update_video_shown(this);
   }
 }
 private static List<ChannelObserver> subscribers =
          new List<ChannelObserver>();
}
```

Observer Pattern: bing-watch scenario

```
interface ChannelObserver {
 void update_video_shown(Channel channel);
}
```

```
class Channel {
 public static void subscribe(ChannelObserver obs) {
   subscribers.Add(obs);
 }
 public static void unsubscribe(ChannelObserver obs) {
   subscribers.Remove(obs);
 }
 public void on_last_video_shown() {
   foreach (ChannelObserver obs in subscribers) {
     observer.update_video_shown(this);
   }
 }
 private static List<ChannelObserver> subscribers =
          new List<ChannelObserver>();
}
```

```
h(Channel) // begin binging channel
 after last video
eo_shown(User)

nnel
```

```
class User: ChannelObserver {
 public void update_video_shown(Channel c) {
   if (c == binged_channel)
     binged_channel = null;
 }
 public void binge_watch(Channel c) {
   binged_channel = c;
 }
 private Channel binged_channel;
}
```

Observer Pattern: update functions

- Having multiple “update_” functions, one for each type of state change, keeps messages **granular**

Observer Pattern: update functions

- Having multiple “update_” functions, one for each type of state change, keeps messages **granular**
  - Observers that do not care about a particular type of update can ignore it (via an empty implementation of the update function)

Observer Pattern: update functions

- Having multiple “update_” functions, one for each type of state change, keeps messages **granular**
  - Observers that do not care about a particular type of update can ignore it (via an empty implementation of the update function)
- Generally it is better to pass the newly-updated data as a parameter to the update function (**push**) as opposed to making observers fetch it each time (**pull**)

Design patterns: takeaways

- Thinking about design before you start coding is usually worthwhile for large projects
  - Design around the most expensive parts of the software engineering process (usually maintenance!)
- Design patterns are reusable solutions to common problems
- Be familiar with them enough to recognize when they’re being used
  - and to know when to use them yourself
  - you can look up details of a pattern if you remember its name!
- Be mindful of and avoid common anti-patterns